Variables

1. Variables in JavaScript

Detailed Explanation:

- **Variables** are containers for storing data values. JavaScript variables are declared using the var, let, or const keywords.
    - **let and const** are block-scoped, meaning they are only accessible within the block they are defined in.
    - **var** is function-scoped, and its usage is generally discouraged in favor of let and const due to hoisting issues.
- **Declaring Variables**:

javascript

Copy code

```
let height;  // Declaration
height = 180; // Initialization
```

- **Constants**: Declared using const. The value assigned to a constant cannot be changed after initialization.

javascript

Copy code

```
const PI = 3.14;  // Declaring a constant
```

Code Snippet:

javascript

Copy code

```
let name = "Stavan";  // String
let age = 35;  // Number
const country = "India";  // Constant


console.log(name);  // Output: Stavan
console.log(age);   // Output: 35
console.log(country);  // Output: India
```

In-depth Questions:

1. **What is the difference between let, const, and var in JavaScript?**
    - **Answer**: let and const are block-scoped, while var is function-scoped. const is used for values that should not be reassigned.
2. **What error do you get when trying to reassign a const variable?**
    - **Answer**: You get an **Uncaught TypeError: Assignment to constant variable**.

Practice Question:

- **True or False**: You can declare a variable without initializing it using let or var.
    - **Answer**: True.

---

2. Variable Scope

Detailed Explanation:

- **Global Scope**: Variables declared outside any function or block are accessible globally.
- **Local/Block Scope**: Variables declared with let or const inside a block ({}) are only accessible within that block.
- **Shadowing**: When a local variable with the same name as a global variable is declared, it "shadows" the global variable within its scope.

Code Snippet:

javascript

Copy code

```
let globalVar = "Global";


{
  let localVar = "Local";
  console.log(localVar);  // Output: Local
}


console.log(globalVar);  // Output: Global
console.log(localVar);  // Error: localVar is not defined
```

In-depth Questions:

1. **What is variable shadowing in JavaScript?**

- **Answer**: Variable shadowing occurs when a local variable has the same name as a global variable, hiding the global variable within that local scope.

2. **Explain the difference between function-scoped and block-scoped variables.**
    - **Answer**: Variables declared with var are function-scoped, meaning they are accessible throughout the entire function. Variables declared with let and const are block-scoped, meaning they are only accessible within the block {} they are declared in.

Practice Question:

- **What will the following code output?**

javascript

Copy code

```
let x = 10;
{
  let x = 5;
  console.log(x);  // ?
}
console.log(x);  // ?
```

Answer: Inside the block, x = 5. Outside the block, x = 10.

---

3. Hoisting

Detailed Explanation:

- **Hoisting**: Variable declarations are "hoisted" (moved) to the top of their scope. However, **only declarations are hoisted**, not initializations.
- Variables declared with var are hoisted to the top of their function or global scope. let and const are also hoisted but remain in a "temporal dead zone" until the execution reaches the line where they are initialized.

Code Snippet:

javascript

Copy code

```
console.log(num);  // Output: undefined
var num = 10;
```

```
console.log(value);  // Error: Cannot access 'value' before initialization

let value = 20;
```

In-depth Questions:

1. **What is the main difference in hoisting behavior between var and let/const?**
    - **Answer**: Variables declared with var are hoisted and initialized with undefined. Variables declared with let and const are hoisted but not initialized, so accessing them before initialization throws an error.
2. **What is the "temporal dead zone" in JavaScript?**
    - **Answer**: The temporal dead zone is the period between entering the scope and the variable being declared where the variable cannot be accessed.

Practice Question:

- **True or False**: In JavaScript, only var declarations are hoisted.
    - **Answer**: False (Both let and const are also hoisted, but they behave differently).

---

4. Changing Variable Values and Type Conversion

Detailed Explanation:

- **Reassigning Variables**: Variables declared with let and var can be reassigned.

javascript

Copy code

```
let steps = 100;

steps = 120;

steps = steps + 200;  // Output: 320
```

- **Type Conversion**: JavaScript automatically converts values between types when necessary. This is known as **type coercion**. For example:

javascript

Copy code

```
let greeting = "Hello";

let count = 100;

greeting = greeting + count;  // Output: "Hello100"
```

Code Snippet:

javascript

Copy code

```
let str = "5";

let num = 10;


let result = str + num;  // Output: "510" (string concatenation)

console.log(result);
```

In-depth Questions:

1. **What is implicit type coercion in JavaScript?**
    - **Answer**: Implicit type coercion is when JavaScript automatically converts a value from one type to another during an operation (e.g., converting a number to a string in concatenation).
2. **What will the following code output?**

javascript

Copy code

```
let x = "50";

let y = 5;

console.log(x - y);
```

- **Answer**: 45 (JavaScript converts the string "50" to a number).

Practice Question:

- **What is the result of 100 + "200" in JavaScript?**
    - **Answer**: "100200" (string concatenation).

---

5. Constants

Detailed Explanation:

- **Constants** are declared using const. Once assigned, their value cannot be changed. Constants must be initialized at the time of declaration.

javascript

Copy code

```
const gravity = 9.8;
```

gravity = 10; // Error: Assignment to constant variable

Code Snippet:

javascript

Copy code

```
const pi = 3.14;

console.log(pi);  // Output: 3.14
```

In-depth Questions:

1. **Can you reassign a const variable after it has been initialized?**
    - **Answer**: No, attempting to reassign a const variable will result in an error.
2. **What is the main difference between const and let?**
    - **Answer**: const variables cannot be reassigned after initialization, while let variables can be reassigned.

Practice Question:

- **True or False**: const must always be initialized at the time of declaration.
    - **Answer**: True.

---

6. Tasks from the File

- **Task 1**: Declare variables for the price and quantity of flowers (roses, lilies, tulips), calculate the total price, and display the results.

javascript

Copy code

```
let priceOfRose = 8, quantityOfRoses = 70;

let priceOfLily = 10, quantityOfLilies = 50;

let priceOfTulip = 2, quantityOfTulips = 120;


let totalRosePrice = priceOfRose * quantityOfRoses;

let totalLilyPrice = priceOfLily * quantityOfLilies;

let totalTulipPrice = priceOfTulip * quantityOfTulips;
```

```
let totalPrice = totalRosePrice + totalLilyPrice + totalTulipPrice;


console.log(`Rose – unit price: ${priceOfRose}, quantity: ${quantityOfRoses}, value: ${totalRosePrice}`);
console.log(`Lily – unit price: ${priceOfLily}, quantity: ${quantityOfLilies}, value: ${totalLilyPrice}`);
console.log(`Tulip – unit price: ${priceOfTulip}, quantity: ${quantityOfTulips}, value: ${totalTulipPrice}`);
console.log(`Total: ${totalPrice}`);
```